

Savings-heuristic algorithm for Travelling-salesman problem

First, all the nodes are sorted in ascending order of total edge-costs from each node. (sum of all column-values in a row of the cost-matrix.)

Then, the savings-heuristic algorithm is run iteratively, using different nodes as the starting-node.

The lowest tour-cost valid-tour generated from all the iterations, is found.

The iterations start, using the sorted nodes-list for starting-node, but also sometimes use randomly-selected starting-nodes to introduce an element of exploration in the search-algorithm strategy.

In certain iterations, the starting-node is also selected from the middle or bottom of the sorted nodes-list.

The same savings-heuristic algoritm works for both euclidean and non-euclidean travelling-salesman problems.

A maximum-iterations-limit (150) and maximum-execution-time-limit (50 seconds) are also set in the script, for handling huge input data-sets and test-cases.

Test-run results for the 8 test-cases given in the validator for travelling-salesman problem.

Test-case runs	0, euc_10	1, non-euc_10	2, euc_25	3, non-euc_25	4, euc_50	5, non-euc_50	6, euc_100	7, non-euc_100
24-8-2026	655.6751331590185	768.988361117	964.812249689	1609.268829199	1340.460459118	3109.62367178	1814.262870419	5688.3586401872
26-8-2026 11:47	655.6751331590184, (7)	754.71076386199, (1)	919.082356725, (17)	1726.442756836, (17)	1319.92637077, (17)	3147.410918719, (6)	1791.15178762, (85)	5746.500427348, (69)
26-8-2026 11:52	655.6751331590184, (5)	693.48129862649, (2)	845.0878576714, (11)	1576.720440208, (9)	1223.4319945288, (48)	2934.84797799, (8)	1699.094389369, (88)	5688.3586401872, (20)
26-8-2026 11:53	655.6751331590184, (5)	673.7787665244, (9)	845.0878576714, (11)	1584.7737918, (4)	1241.6814249, (20)	2995.65760728, (33)	1736.687648228, (34)	5635.42056207, (0)
26-8-2026 11:54	655.6751331590184, (5)	673.7787665244, (9)	787.5752663189, (24)	1576.720440208, (9)	1223.4572162, (29)	2934.84797799, (8)	1707.964491164, (59)	5591.8855703, (3)
26-8-2026 12:16; max-iterations=50	655.6751331590184, (5)	673.7787665244, (9)	787.5752663189, (24)	1576.720440208, (9)	1196.460993216, (26)	2934.84797799, (8)	1693.69099236, (22)	5622.59455523, (81)
26-8-2026 12:17; max-iterations=100	655.6751331590184, (5)	673.7787665244, (9)	787.5752663189, (24)	1576.720440208, (9)	1196.460993216, (26)	2934.84797799, (8)	1686.784307951, (99)	5591.8855703, (3)

The starting-node selected for each tour-generation in the above results, is mentioned inside paranthesis next to each tour-cost.

It is observed in the test-runs listed above, that the lowest-cost valid-tours are found after greater number of iterations on different starting-nodes.

The best and lowest-cost valid-tours for all test-cases are found in the final test-run, where the savings-heuristic algorithm was iteratively run with all given nodes as the starting-node in each test-case. (The maximum-iteration limit was set to 100 in the final test-run listed above.)

These results indicate that the node with the lowest total-cost to all other nodes, (node closest to the geometric-center of all the node-locations in euclidean t.s.p.), is not necessarily the best starting-node for generating the lowest-cost valid-tour using the savings-heuristic algorithm.

Savings-heuristic algorithm logic :

After the starting-node is selected, all the edges between all the other nodes are sorted in decreasing-order of edge-cost savings compared to edges with the starting-node.

$$(\text{cost-savings of edge i-to-j}) = (\text{cost of i-to-startnode}) + (\text{cost of startnode-to-j}) - (\text{cost of i-to-j})$$

Then the valid-tour is generated by adding the highest cost-saving edges one by one, without creating sub-loops or closing the tour prematurely.

The total cost of the generated valid-tour is compared to find the minimum-cost valid-tour for each test-case.